

SQL Cheat Sheet — Examen

Sintaxis esencial: Consultas, Funciones de Agregación, Joins, Window Functions, Fechas y Texto en PostgreSQL

1. Consultas y Filtrado Básico

COUNT

Contar el número de registros.

```
SELECT COUNT(*) AS total_peliculas
FROM peliculas
WHERE genero = 'Drama';
```

SELECT DISTINCT

Mostrar valores únicos sin duplicados.

```
SELECT DISTINCT genero, anio
FROM peliculas
ORDER BY genero;
```

BETWEEN

Filtrar dentro de un rango numérico.

```
SELECT titulo, presupuesto
FROM peliculas
WHERE presupuesto BETWEEN 300 AND 1000;
```

IN

Filtrar dentro de una lista de valores.

```
SELECT titulo, genero
FROM peliculas
WHERE genero IN ('Drama', 'Triste', 'Comedia');
```

LIKE

Buscar patrones en texto (%).

```
SELECT titulo
FROM peliculas
WHERE titulo LIKE 'T%'
    OR titulo LIKE '%amor%';
```

IS NULL

Filtrar registros con valores nulos.

```
SELECT titulo, director
FROM peliculas
WHERE director IS NULL;
```

ROUND

Redondear decimales.

```
SELECT genero, ROUND(AVG(presupuesto), 2) AS media
FROM peliculas
GROUP BY genero;
```

ORDER BY

Ordenar resultados (ASC / DESC).

```
SELECT titulo, duracion
FROM peliculas
ORDER BY duracion DESC, titulo ASC;
```

GROUP BY

Agrupar filas para funciones de agregación.

```
SELECT genero, COUNT(*) AS num_peliculas,
    ROUND(AVG(presupuesto), 2) AS media
FROM peliculas
GROUP BY genero;
```

HAVING

Filtrar los grupos de un GROUP BY.

```
SELECT genero, COUNT(*) AS num_peliculas
FROM peliculas
GROUP BY genero
HAVING COUNT(*) > 1
ORDER BY num_peliculas DESC;
```

2. Joins y Combinación de Tablas

INNER JOIN

Une tablas mostrando solo las coincidencias.

```
SELECT p.titulo, a.nombre AS actor
FROM peliculas AS p
JOIN actuaciones AS a ON p.id = a.id_pelicula;
```

Aliases (AS)

Abreviar nombres de tablas para limpieza.

```
SELECT p.titulo, p.presupuesto
FROM peliculas AS p
WHERE p.presupuesto > 500;
```

USING

Unir tablas por columna con idéntico nombre.

```
SELECT p.titulo, a.nombre
FROM peliculas AS p
JOIN actuaciones AS a USING (id_pelicula);
```

LEFT JOIN

Todo de la izquierda y coincidencias de la derecha.

```
SELECT p.titulo, a.nombre
FROM peliculas AS p
LEFT JOIN actuaciones AS a ON p.id = a.id_pelicula
WHERE a.nombre IS NULL;
```

RIGHT JOIN

Todo de la derecha y coincidencias de la izquierda.

```
SELECT p.titulo, a.nombre
FROM peliculas AS p
RIGHT JOIN actuaciones AS a ON p.id = a.id_pelicula;
```

FULL JOIN

Todo de ambas tablas (con nulos si falta match).

```
SELECT p.titulo, a.nombre
FROM peliculas AS p
FULL JOIN actuaciones AS a ON p.id = a.id_pelicula;
```

CROSS JOIN

Producto cartesiano (combina cada fila con todas).

```
SELECT p.titulo, s.nombre_sala
FROM peliculas AS p
CROSS JOIN salas AS s;
```

SELF JOIN

Unir una tabla consigo misma usando alias.

```
SELECT e1.nombre AS empleado, e2.nombre AS jefe
FROM empleados AS e1
JOIN empleados AS e2 ON e1.jefe_id = e2.id
ORDER BY e2.nombre;
```

3. Operaciones de Conjuntos y Subconsultas

UNION

Une resultados verticales eliminando duplicados.

```
SELECT nombre FROM clientes_madrid
UNION
SELECT nombre FROM clientes_barcelona;
```

UNION ALL

Une resultados verticales conservando duplicados.

```
SELECT nombre FROM clientes_2023
UNION ALL
SELECT nombre FROM clientes_2024;
```

INTERSECT

Muestra solo los registros comunes a ambas consultas.

```
SELECT id_cliente FROM pedidos_2023
INTERSECT
SELECT id_cliente FROM pedidos_2024;
```

EXCEPT

Registros de la 1ª consulta que no están en la 2ª.

```
SELECT id_cliente FROM clientes
EXCEPT
SELECT id_cliente FROM pedidos;
```

Subconsulta (WHERE)

Filtrar usando el resultado de otra consulta.

```
SELECT titulo
FROM peliculas
WHERE id IN (
    SELECT id_pelicula FROM premios WHERE anio = 2023
);
```

Subconsulta (FROM)

Tratar una consulta interna como tabla temporal.

```
SELECT sub.genero, sub.media
FROM (
    SELECT genero, AVG(presupuesto) AS media
    FROM peliculas
    GROUP BY genero
) AS sub
WHERE sub.media > 500;
```

Subconsulta correlacionada

Filtrar usando un cálculo que depende de cada fila externa.

```
SELECT nombre, sueldo, departamento
FROM empleados e
WHERE sueldo > (
    SELECT AVG(sueldo) FROM empleados
    WHERE departamento = e.departamento
);
```

CTE (WITH)

Crear una mini tabla temporal antes de consultar.

```
WITH ventas_totales AS (
    SELECT vendedor, SUM(importe) AS total
    FROM ventas
    GROUP BY vendedor
)
SELECT * FROM ventas_totales
WHERE total > 10000
ORDER BY total DESC;
```

4. Funciones de Ventana y Agrupaciones Avanzadas

OVER()

Mostrar un cálculo global al lado de cada fila.

```
SELECT nombre, sueldo,
       AVG(sueldo) OVER() AS media_general
FROM empleados;
```

PARTITION BY

Calcular el OVER() dividido por grupos.

```
SELECT nombre, departamento, sueldo,
       AVG(sueldo) OVER(PARTITION BY departamento) AS media_dep
FROM empleados;
```

LAG()

Muestra el valor de la fila anterior.

```
SELECT fecha, ventas,
       LAG(ventas) OVER(ORDER BY fecha) AS ventas_ayer
FROM ventas_diarias;
```

LEAD()

Muestra el valor de la fila siguiente.

```
SELECT fecha, ventas,
       LEAD(ventas) OVER(ORDER BY fecha) AS ventas_manana
FROM ventas_diarias;
```

FIRST_VALUE()

Muestra el valor de la primera fila del grupo.

```
SELECT nombre, departamento, sueldo,
       FIRST_VALUE(nombre) OVER (
           PARTITION BY departamento ORDER BY sueldo DESC
       ) AS mejor_pagado
FROM empleados;
```

RANK()

Ranking con saltos de números si hay empates.

```
SELECT nombre, puntos,
       RANK() OVER(ORDER BY puntos DESC) AS posicion
FROM jugadores;
```

DENSE_RANK()

Ranking compacto sin saltos si hay empates.

```
SELECT nombre, puntos,
       DENSE_RANK() OVER(ORDER BY puntos DESC) AS posicion
FROM jugadores;
```

NTILE(3)

Divide los datos en 3 grupos iguales (tercios).

```
SELECT nombre, sueldo,
       NTILE(3) OVER(ORDER BY sueldo) AS tercio
FROM empleados;
```

CASE WHEN

Condiciones lógicas (Si / Entonces).

```
SELECT nombre, sueldo,
       CASE WHEN sueldo > 3000 THEN 'Alto'
            WHEN sueldo BETWEEN 1500 AND 3000 THEN 'Medio'
            ELSE 'Bajo'
       END AS categoria
FROM empleados;
```

ROLLUP

Subtotales jerárquicos en cascada + gran total global.

```
SELECT pais, ciudad, SUM(ventas) AS total
FROM ventas
GROUP BY ROLLUP (pais, ciudad)
ORDER BY pais, ciudad;
```

CUBE

Todas las combinaciones posibles de subtotales + gran total.

```
SELECT pais, ciudad, SUM(ventas) AS total
FROM ventas
GROUP BY CUBE (pais, ciudad)
ORDER BY pais, ciudad;
```

5. Extensiones y Pivotes (PostgreSQL)

CREATE EXTENSION

Activa extensiones o módulos adicionales en PostgreSQL (necesario para tablefunc).

```
CREATE EXTENSION tablefunc;
```

CROSSTAB()

Convierte filas en columnas (tabla pivote). Requiere extensión tablefunc.

```
SELECT * FROM crosstab(
'SELECT vendedor, mes, ventas
FROM ventas_mensuales ORDER BY 1,2',
'SELECT DISTINCT mes FROM ventas_mensuales ORDER BY 1'
) AS res (vendedor TEXT, enero INT, febrero INT, marzo INT);
```

6. Marcos de Ventana (Frames: ROWS BETWEEN)

PRECEDING

Mira hacia atrás (filas anteriores).

```
2 PRECEDING // 2 filas atrás
```

CURRENT ROW

La fila actual en la que está posicionado el cursor.

```
CURRENT ROW
```

FOLLOWING

Mira hacia adelante (filas siguientes).

```
1 FOLLOWING // 1 fila siguiente
```

UNBOUNDED

Significa "sin límite" o infinito. Siempre va con PRECEDING o FOLLOWING.

```
UNBOUNDED PRECEDING // Desde el principio
```

UNBOUNDED PRECEDING AND CURRENT ROW

Suma acumulada clásica (por defecto si no hay marco).

```
SUM(v) OVER (ORDER BY fecha
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
```

2 PRECEDING AND CURRENT ROW

Suma móvil de 3 periodos (actual + 2 anteriores).

```
SUM(v) OVER (ORDER BY día
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
```

CURRENT ROW AND UNBOUNDED FOLLOWING

Ventana hacia el futuro. ¡Vital para LAST_VALUE()!

```
LAST_VALUE(puntos) OVER (ORDER BY puntos DESC
ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING)
```

1 PRECEDING AND 1 FOLLOWING

Ventana centrada de 3 filas (anterior, actual y siguiente).

```
AVG(temp) OVER (ORDER BY hora
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
```

7. Arrays, Fechas y Funciones de Texto

ANY

Comprueba si un valor está en un array.

```
SELECT titulo FROM peliculas
WHERE 'rojo' = ANY(colores);
```

@>

Comprueba si un array contiene otros elementos.

```
SELECT titulo FROM peliculas
WHERE colores @> ARRAY['rojo','azul'];
```

INTERVAL

Sumar o restar tiempo a una fecha.

```
SELECT fecha_estreno,
       fecha_estreno + INTERVAL '3 days' AS fecha_limite
FROM peliculas;
```

EXTRACT

Sacar el año, mes o día de una fecha.

```
SELECT titulo, EXTRACT(YEAR FROM fecha_estreno) AS anio
FROM peliculas;
```

DATE_TRUNC

Redondear una fecha al mes o año inicial.

```
SELECT fecha, DATE_TRUNC('month', fecha) AS mes_inicio
FROM ventas;
```

CONCAT / ||

Unir varios textos en uno solo.

```
SELECT nombre || ' ' || apellido AS nombre_completo
FROM clientes;
```

UPPER / LOWER

Cambiar texto a mayúsculas o minúsculas.

```
SELECT UPPER(nombre) AS nombre_mayus,
       LOWER(email) AS email_minus
FROM clientes;
```

REPLACE

Cambiar una parte del texto por otra.

```
SELECT REPLACE(texto, 'viejo', 'nuevo') AS actualizado
FROM comentarios;
```

LENGTH

Contar los caracteres de un texto.

```
SELECT titulo, LENGTH(titulo) AS num_caracteres
FROM peliculas;
```

SUBSTRING

Cortar un trozo de texto.

```
SELECT SUBSTRING(texto, 1, 5) AS resumen
FROM comentarios;
```

TRIM

Quitar espacios vacíos de los extremos.

```
SELECT TRIM(columna) AS columna_limpia
FROM tabla;
```

8. Búsqueda Full-Text y Ortografía (PostgreSQL)

to_tsvector()

Prepara y limpia el texto (quita palabras vacías como "el", "de" y normaliza raíces).

```
SELECT to_tsvector('spanish', contenido)
FROM articulos;
```

to_tsquery()

Procesa el término o palabra clave que introduce el usuario en el buscador.

```
SELECT to_tsquery('spanish', 'programación & sql');
```

Operador @@

Conecta ambos mundos: comprueba si el texto coincide con la consulta inteligente.

```
SELECT titulo FROM articulos
WHERE to_tsvector('spanish', contenido)
@@ to_tsquery('spanish', 'programación');
```

levenshtein()

Mide la distancia ortográfica (número de errores/cambios) entre dos palabras.

```
SELECT nombre FROM productos
WHERE levenshtein(nombre, 'iphone') <= 2;
```

9. Ejercicios Resueltos — Subconsultas y CTE

Ejercicio 1 — Subconsulta larga con CTE + correlación

Para cada empleado, calcular cuánto gana por encima (o por debajo) de la media de su departamento, cuántos compañeros tiene, y quedarse solo con los que superan esa media, ordenados por departamento y por la mayor diferencia.

```
-- Paso 1: calculamos la media y el n° de empleados por departamento (CTE)
WITH medias_departamento AS (
    SELECT
        departamento,
        ROUND(AVG(sueldo), 2) AS media_departamento,
        COUNT(*) AS num_empleados
    FROM empleados
    GROUP BY departamento
)

-- Paso 2: unimos cada empleado con la media de su departamento
SELECT
    e.nombre,
    e.departamento,
    e.sueldo,
    m.media_departamento,
    ROUND(e.sueldo - m.media_departamento, 2) AS diferencia,
    m.num_empleados
FROM empleados AS e
JOIN medias_departamento AS m
    ON e.departamento = m.departamento

-- Paso 3: nos quedamos solo con quienes ganan más que la media de su departamento
WHERE e.sueldo > m.media_departamento
ORDER BY e.departamento, diferencia DESC;
```

Ejercicio 1b — La misma idea con subconsulta correlacionada (sin CTE)

Versión equivalente usando una subconsulta correlacionada dentro del WHERE: por cada fila de "e", se vuelve a calcular la media de su departamento.

```
SELECT
    e.nombre,
    e.departamento,
    e.sueldo,
    (
        SELECT ROUND(AVG(e2.sueldo), 2)
        FROM empleados AS e2
        WHERE e2.departamento = e.departamento
    ) AS media_departamento
FROM empleados AS e
WHERE e.sueldo > (
    SELECT AVG(e2.sueldo)
    FROM empleados AS e2
    WHERE e2.departamento = e.departamento
)
ORDER BY e.departamento, e.sueldo DESC;
```

9. Ejercicios Resueltos — Ventanas y CASE WHEN

Ejercicio 2 — Funciones de ventana combinadas

Para cada vendedor y mes: total acumulado de ventas, comparación con el mes anterior (LAG), variación porcentual, y ranking dentro de su región cada mes.

```
SELECT
    vendedor,
    region,
    mes,
    ventas,

    -- Acumulado de ventas del vendedor mes a mes
    SUM(ventas) OVER (
        PARTITION BY vendedor
        ORDER BY mes
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS acumulado_vendedor,

    -- Ventas del mes anterior del mismo vendedor
    LAG(ventas) OVER (PARTITION BY vendedor ORDER BY mes) AS ventas_mes_anterior,

    -- Variación absoluta y porcentual respecto al mes anterior
    ROUND(ventas - LAG(ventas) OVER (PARTITION BY vendedor ORDER BY mes), 2)
        AS variacion_absoluta,
    ROUND(
        100.0 * (ventas - LAG(ventas) OVER (PARTITION BY vendedor ORDER BY mes))
        / NULLIF(LAG(ventas) OVER (PARTITION BY vendedor ORDER BY mes), 0), 2
    ) AS variacion_pct,

    -- Ranking del vendedor dentro de su región, cada mes
    RANK() OVER (PARTITION BY region, mes ORDER BY ventas DESC) AS ranking_region,

    -- Media móvil de 3 meses (mes actual + 2 anteriores)
    ROUND(AVG(ventas) OVER (
        PARTITION BY vendedor
        ORDER BY mes
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ), 2) AS media_movil_3m

FROM ventas_mensuales
ORDER BY region, mes, ranking_region;
```

Ejercicio 3 — CASE WHEN de varios niveles

Clasificar a cada cliente en un segmento según su gasto total Y su antigüedad, combinando varias condiciones con AND / OR dentro del CASE.

```
SELECT
    cliente_id,
    nombre,
    gasto_total,
    antiguedad_anios,
    CASE
        WHEN gasto_total >= 5000 AND antiguedad_anios >= 3
            THEN 'VIP Fiel'
        WHEN gasto_total >= 5000 AND antiguedad_anios < 3
            THEN 'VIP Nuevo'
        WHEN gasto_total BETWEEN 1000 AND 4999 AND antiguedad_anios >= 3
            THEN 'Habitual Fiel'
        WHEN gasto_total BETWEEN 1000 AND 4999
            THEN 'Habitual'
        WHEN gasto_total < 1000 AND antiguedad_anios >= 3
            THEN 'Bajo Consumo Fiel'
        ELSE 'Ocasional'
    END AS segmento_cliente,

    -- Un segundo CASE anidado en la misma consulta, para prioridad de contacto
    CASE
        WHEN gasto_total >= 5000 THEN 'Alta'
        WHEN gasto_total >= 1000 THEN 'Media'
        ELSE 'Baja'
    END AS prioridad_contacto

FROM clientes
ORDER BY gasto_total DESC;
```

Ejercicio 4 — Combinado: JOIN + GROUP BY + HAVING + CASE + ORDER BY

Ingresos totales por película y por sala, solo de las películas con más de 2 sesiones, etiquetando cada película según si supera o no la media general de ingresos.

```
WITH ingresos_pelicula AS (  
  SELECT  
    p.titulo,  
    s.nombre_sala,  
    COUNT(ss.id)           AS num_sesiones,  
    SUM(ss.entradas_vendidas * ss.precio_entrada) AS ingresos_totales  
  FROM peliculas AS p  
  JOIN sesiones AS ss ON ss.id_pelicula = p.id  
  JOIN salas AS s      ON s.id = ss.id_sala  
  GROUP BY p.titulo, s.nombre_sala  
  HAVING COUNT(ss.id) > 2  
)  
SELECT  
  titulo,  
  nombre_sala,  
  num_sesiones,  
  ingresos_totales,  
  CASE  
    WHEN ingresos_totales > (SELECT AVG(ingresos_totales) FROM ingresos_pelicula)  
    THEN 'Por encima de la media'  
    ELSE 'Por debajo de la media'  
  END AS comparacion_media  
FROM ingresos_pelicula  
ORDER BY ingresos_totales DESC;
```